Architectural Justification: Self-Service Coffee Kiosk System

We chose Layered Architecture for the Self-Service Coffee Kiosk System.

Architectural Choice

The system is structured using a Layered Architecture, organized into four layers: a Presentation layer (Touchscreen UI), a Business Logic layer (Order Manager, Payment Service), a Data layer (Menu & Pricing Data), and a Hardware Integration layer (Receipt Printer Interface). Each layer communicates only with the layer directly adjacent to it, keeping responsibilities cleanly separated.

Two Reasons for This Choice

1. Single-device scope: The kiosk is a single, self-contained device with a fixed, well-defined workflow (select drink, choose size, pay, print receipt). A Layered Architecture matches this scope precisely, offering clear separation of concerns without the operational overhead of independently deployed services that a Microservices approach would introduce for no real benefit here.

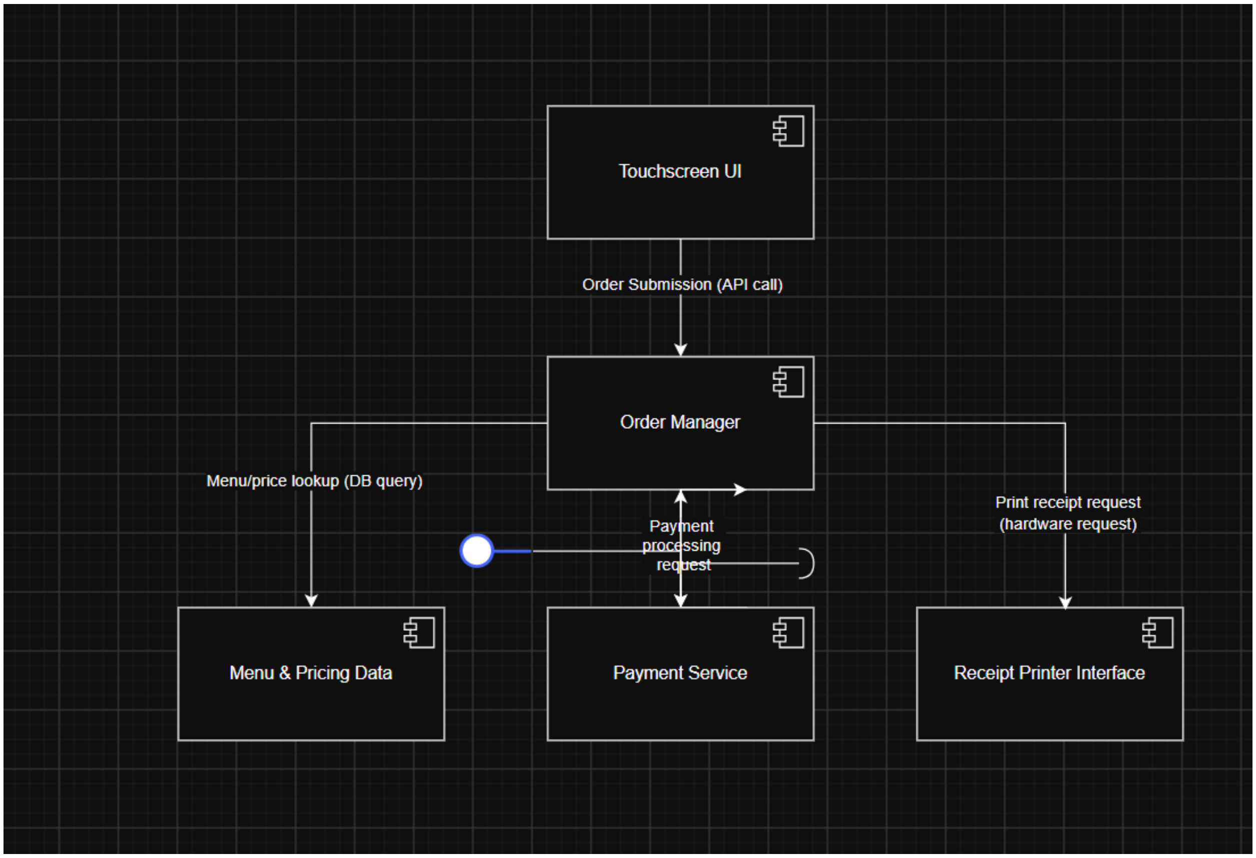
2. Predictable, sequential interactions: Order placement, payment, and receipt printing happen in a fixed sequence for a single customer at a time. This maps naturally onto layers calling the layer below them, and avoids the added network latency and distributed-system complexity that a Microservices or multi-node Client-Server style would add without a corresponding need for independent scaling.

Security Advantage

Because payment handling is isolated inside the Payment Service component within the Business Logic layer, the Touchscreen UI (Presentation layer) never has direct access to card data or payment credentials — it only communicates through the Order Manager's well-defined interface. This layering limits the attack surface: a compromise of the UI layer cannot directly expose payment processing internals, since all sensitive operations are encapsulated behind the Payment Service's provided interface.

Performance Benefit

Since all layers run locally on the same kiosk device, calls between the Touchscreen UI, Order Manager, Payment Service, Menu & Pricing Data, and Receipt Printer Interface avoid the network round-trip latency inherent to Microservices or remote Client-Server calls. This keeps the full select-pay-print flow fast and responsive, which is critical for the under-60-second order completion expected at a busy café kiosk.


Touchscreen UI
Order Submission (API call)
Order Manager
Menu/price lookup (DB query)
Payment processing request
Print receipt request (hardware request)
Menu & Pricing Data
Payment Service
Receipt Printer Interface